0/1 Knapsack

Topic: DynamicProgramming File: dp2.pdf

PROBLEM

Each of n items has a weight and value. Choose items without splitting them so total weight is at most capacity W and value is maximized.

STATE

dp[i][w] is the best value using the first i items with capacity w.

TRANSITION

If item i is too heavy, copy dp[i-1][w]. Otherwise take max(dp[i-1][w], value[i] + dp[i-1][w-weight[i]]).

EXAMPLE

Items (weight,value): (2,4), (3,5), (4,6), capacity 5. Taking the first two items gives total weight 5 and value 9.

COMPLEXITY

Standard table: O(nW) time and O(nW) space. A one-dimensional descending-capacity update reduces space to O(W).

IMPORTANT

This is pseudo-polynomial because the running time depends on the numeric capacity W, not only the number of input bits.